Davante

14

ENTORNOS DE DESARROLLO

Procesos de ejecución de pruebas

ÍNDICE

/ 1. Introducción y contextualización práctica 3

/ 2. Enfoques de pruebas 4

/ 3. Tipos de pruebas basadas en cajas 4

/ 4. Caso práctico 1: “Elección de la prueba más adecuada” 5

/ 5. Niveles de pruebas: Usabilidad, accesibilidad y aceptación 5

/ 6. Niveles de pruebas: Seguridad y A/B 6

/ 7. Caso práctico 2: “Pruebas A/B” 7

/ 8. Resumen y resolución del caso práctico de la unidad 8

/ 9. Bibliografía 8

© MEDAC 978-84-18983-00-9
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Comprender la importancia de ejecución de pruebas.

Conocer los diferentes tipos de enfoque de pruebas.

Comprender los diferentes métodos de prueba de cajas.

Conocer los tipos de pruebas fundamentales.

/ 1. Introducción y contextualización práctica

A la hora desarrollar una aplicación, es necesario poder implementar toda la funcionalidad que nos pide nuestro usuario o cliente final. Para poder comprobar que el *software* que hemos desarrollado cumple con los requisitos que nos han solicitado, es necesario tener un conjunto de herramientas que lo permitan.

Para ello, se introducen las pruebas en el desarrollo *software*. A priori, la realización de las pruebas puede parecer algo trivial y secundario, de hecho, así se ha considerado durante mucho tiempo. Sin embargo, el desarrollo y ejecución de las pruebas tiene cada vez más relevancia, ya que muchas de las metodologías utilizadas en la actualidad se basan en las pruebas para poder avanzar más rápidamente. Por ello, no es de extrañar que el tiempo que podemos dedicar a las pruebas, en ocasiones, pueda superar al tiempo que requerimos para poder implementar una funcionalidad.

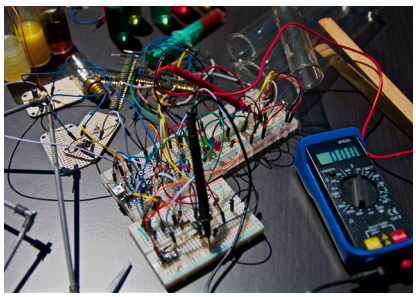


Fig. 1. Las pruebas pueden ser algo caóticas si no se sigue el proceso.

En este tema, vamos a ver diferentes enfoques para el desarrollo de las pruebas, diferentes metodologías y algunas técnicas que nos permitirán realizar pruebas de una manera más eficiente para el *software* que desarrollamos.

Escucha el siguiente audio en el que planteamos el caso práctico que iremos resolviendo a lo largo de esta unidad.

Audio intro. "Accesibilidad en posicionamiento web"
https://bit.ly/3qkfLmL

/ 2. Enfoques de pruebas

A la hora de realizar las pruebas del código, es necesario tener en cuenta diferentes enfoques para poder cubrir el mayor número de líneas de código. Podríamos decir que existen dos grandes grupos de enfoque para la realización de pruebas: el enfoque exploratorio y el enfoque basado en cajas.

A. **Enfoque exploratorio**

Permite definir pruebas de una manera personal, libre y bajo la responsabilidad del diseñador de las pruebas para llevar a cabo la validación del sistema. En este enfoque no existe una metodología como tal para poder aplicar sistemáticamente las pruebas. Además, desde el punto de vista de la ejecución y resultados de las pruebas, se deja libre interpretación al diseñador de estas. En este tipo de pruebas, podemos observar que es muy complicado establecer una generalización que podamos utilizar en diferentes proyectos de una manera sencilla.

B. **Enfoque basado en cajas**

Es uno de los más utilizados tradicionalmente a la hora de realizar pruebas de un sistema. Podríamos decir que los diseñadores de las pruebas realizarán estas acorde con la información que tengan del sistema. Esto quiere decir que a más información del sistema, las pruebas estarán más dirigidas a puntos en concreto. Podemos distinguir diferentes tipos de enfoques a la hora de realizar las pruebas de cajas: pruebas de caja negra, blanca y/o gris. Nótese que los diferentes enfoques de cajas no son incompatibles entre sí, pudiéndose combinar libremente.



Fig. 2. Distintos enfoques para las pruebas.

Vídeo 1. "Pruebas exploratorias y basadas en cajas"
https://bit.ly/33CINUZ

/ 3. Tipos de pruebas basadas en cajas

Como hemos visto anteriormente, existen diferentes tipos de pruebas de cajas. A continuación, realizaremos una descripción de cada una de ellas:

- **Pruebas de caja blanca.** Al realizar este tipo de pruebas, el nivel de conocimiento de los detalles de implementación del sistema es muy alto. Por ello, al realizar las pruebas, se dirigen a una determinada parte del código. Estas pruebas requieren un cierto nivel de conocimiento de programación para poder lanzarlas y dirigirlas a estas partes de interés. Es posible utilizar este enfoque para crear pruebas unitarias, de integración, etc.

- **Pruebas de caja negra.** Este tipo de pruebas, también conocido como pruebas funcionales, permiten realizar pruebas del *software* sin conocer detalles de implementación. Se dice que el *software* se observa como una caja negra de la cual no se tiene ningún tipo de información y, simplemente, se realizan pruebas de las diferentes funcionalidades que debería de implementar. Normalmente, para realizar este tipo de pruebas, nos basamos en la especificación del sistema para saber las funcionalidades que son requeridas. Para poder tener una idea clara de estas funcionalidades, nos podemos dirigir directamente a los casos de uso y a los requisitos de las etapas iniciales del proyecto. Para comprobar que una prueba se ha superado correctamente, tendremos que ejecutarla, ver el resultado obtenido y compararlo con lo que se espera en los casos de uso. Esta es una de las razones por la que es sumamente importante que los casos de uso queden definidos en las etapas iniciales de forma clara.

Audio 1. "Pruebas estructurales"
https:/bit.ly/2LfOGxf

- **Pruebas de caja gris.** Este tipo de pruebas surge al tener una combinación de las pruebas de caja blanca y las pruebas de caja negra. El objetivo de estas pruebas es intentar encontrar los posibles errores debido a la existencia de una mala estructura o uso incorrecto en las aplicaciones. Para ejecutar este tipo de pruebas, es necesario conocer la documentación que describa la aplicación, así como un pequeño detalle de implementación.

Audio 2. "Combinación de pruebas de cajas"
https://bit.ly/2KZS0Ak

/ 4. Caso práctico 1: "Elección de la prueba más adecuada"

Planteamiento: En nuestro equipo de desarrollo, necesitamos comprobar determinadas funcionalidades del software que estamos a punto de sacar a producción.

Hemos realizado unos pequeños cambios en una función de código y solamente queremos realizar pruebas para probar su comportamiento.

Nudo: ¿Cuál crees que sería el mejor enfoque?

Desenlace: Según lo que hemos visto en el tema, para poder probar una determinada funcionalidad, lo más común es que tengamos detalles del código.

Además, en el caso, se expone que los cambios que se han realizado únicamente afectan a una función de código. Junto a esto, se nos comenta que solo queremos probar dichos cambios.

Considerando todo, el mejor enfoque para poder probar los nuevos cambios sería mediante la utilización de pruebas de caja blanca. Gracias a ellas, podemos conocer los detalles de implementación y dirigir nuestras pruebas a las partes del código que hayan sido modificadas y tener mejor cobertura en las pruebas que realicemos.



Fig. 3. Las pruebas de caja blanca permiten ver todo el código.

/ 5. Niveles de pruebas: Usabilidad, accesibilidad y aceptación

De manera general, podríamos decir que existen al menos tres niveles para la ejecución de las pruebas: unitarias, de integración y de sistemas.

Además, si consideramos las nuevas tendencias, podríamos hablar de un 4º nivel que incluiría las pruebas de aceptación.

Dentro de cada nivel de prueba, podemos distinguir diferentes tipos de técnicas y tácticas que se pueden aplicar para detectar posibles errores en el *software*. A continuación, vamos a describir algunas tendencias de las técnicas más utilizadas en la actualidad.

Nótese que no todas las técnicas que vemos pertenecen al mismo nivel de ejecución de pruebas.

A. **Pruebas de usabilidad y accesibilidad**

Las pruebas de usabilidad tratan de comprobar si la interfaz de usuario es fácil de utilizar y entendible por los usuarios que acceden a ella.

Es un tipo de prueba que puede ser relativamente complicada de ejecutar porque no se puede automatizar como sucede con otros tipos. Por ello, se dice que son pruebas que requieren de la interacción humana para poder validarlas y obtener resultados.

Las pruebas de usabilidad se suelen confundir con las pruebas de accesibilidad. Sin embargo, las pruebas de accesibilidad se centran en comprobar si el *software* que hemos desarrollado es accesible por usuarios que puedan tener alguna limitación sensorial. Para ello, existen diferentes estándares que permiten determinar las reglas para saber si un *software* es accesible o no.

En el caso de las páginas web, podemos encontrar herramientas que miden la accesibilidad de nuestra web, otorgando una calificación que puede ser utilizada en el SEO.

B. **Pruebas de aceptación**

Este tipo de pruebas se ubican en las últimas etapas previas a la liberación del software y se centran en determinar si este cumple todas las necesidades que hemos incluido en nuestra etapa de captura de requisitos. Además, podríamos decir que las pruebas de aceptación se extienden más en el tiempo, ya que consideran los problemas que tienen los usuarios a lo largo de la ejecución del código. Normalmente, pueden tener en cuenta datos estadísticos, comportamiento de la aplicación a través de blogs, encuestas, etc.



Vídeo 2. "Pruebas de caja negra"
https:/bit.ly/2Ltv1KH

/ 6. Niveles de pruebas: Seguridad y A/B

A. **Pruebas de seguridad**

Normalmente, son esenciales en cualquier desarrollo *software* que pueda requerir el tratamiento de cualquier dato sensible. La idea es poder proteger al usuario y a nuestro sistema de posibles ataques externos. Existen determinadas ISOs para poder determinar qué tipo de información debe ser protegida y el nivel de seguridad que se debe aplicar.

Las aplicaciones que desarrollamos tienen que considerar el tipo de datos y actuar en consecuencia. Por ejemplo, no será lo mismo que nuestra aplicación almacene correos de usuarios que sus expedientes médicos. En este último caso, el nivel de seguridad que se aplicaría tendría que ser muy elevado, pues la información que contiene es muy sensible y personal.

B. **Pruebas A/B**

Este tipo de pruebas permite desarrollar una metodología en la que tenemos dos componentes que comparar. Una posibilidad es que tengamos dos equipos de desarrollo trabajando en la misma solución y comparemos cuál produce mejores resultados. Para ello, podemos lanzar pruebas de este estilo y tener un resultado a *posteriori*.

En otras ocasiones, podemos tener una versión previa a la solución del error y compararla con la solución en la que el error se ha solucionado. Este tipo de pruebas suele ser muy utilizada en el desarrollo de aplicaciones móviles dado que existe un amplio número de dispositivos diferentes, así como tecnologías de desarrollo. Tras un procedimiento de cambio de tecnología, es normal que se realice un proceso de comparación entre la anterior y la nueva implementación.



Fig. 4. El proceso de prueba requiere sincronización.

Audio 3. "Proceso de pruebas"
https://bit.ly/3fYoXID

/ 7. Caso práctico 2: "Pruebas A/B"

Planteamiento: Nuestro *software* más utilizado sufre una gran caída a nivel mundial y necesitamos incluir un mayor número de recursos para poder averiguar qué está pasando. Por ello, ponemos a tres equipos a trabajar en paralelo para que ofrezcan diferentes tipos de soluciones.

El problema nos surge a la hora de validar la solución del equipo que mejor resuelve el problema.

Nudo: ¿Qué tipo de pruebas harías? ¿Con qué solución te quedarías?

Desenlace: Como hemos visto en el tema, existen diferentes enfoques para poder probar diferentes soluciones *software* de los equipos que tengamos trabajando en paralelo.

Gracias a las pruebas de tipo A/B, podríamos plantear una metodología de pruebas por pares en la que siempre comprobemos la solución actual con la solución propuesta. Cuando tengamos un mejor resultado de la solución propuesta en comparación con la solución existente, podemos pasar dicha propuesta como una candidata a la solución final.

Esto lo podemos repetir con cada una de las soluciones que tengamos y solo pasarán a ser candidatas aquellas que mejoren los resultados con respecto a la solución existente. En el caso de tener una única candidata, esta será la solución final.

Si llegamos a tener más de una candidata, repetiremos el proceso de prueba y siempre nos quedaremos con la que tenga mejor resultado. Repetiremos este proceso hasta que solo tengamos una candidata.

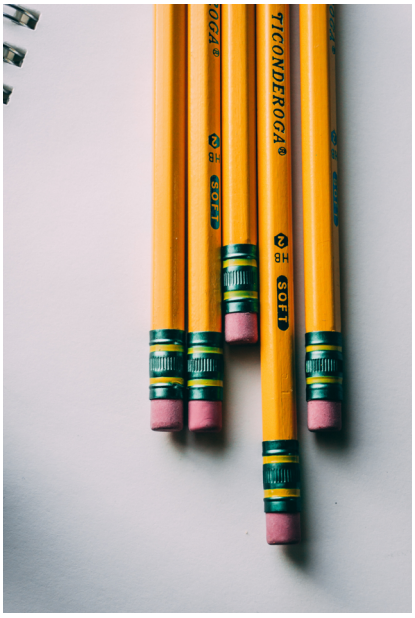


Fig. 5. Debemos elegir la mejor solución candidata.

/ 8. Resumen y resolución del caso práctico de la unidad

En este tema, hemos presentado **diferentes enfoques** para poder realizar las pruebas centrándonos en las pruebas funcionales, atendiendo al uso de las **pruebas de cajas.** Sin embargo, como hemos podido observar, no se trata de las únicas pruebas disponibles que podemos lanzar en un sistema, pues existe otro tipo que debemos tener en cuenta para poder aumentar la calidad de nuestro *software* y llegar a mayor número de usuarios.

Un ejemplo de ello son las **pruebas de usabilidad y accesibilidad,** que, tradicionalmente, han tenido poca importancia. En la actualidad, podemos ver que estas pruebas son cada vez más importantes porque determinan lo atrayente que puede ser una aplicación para el consumidor final.

Las pruebas de accesibilidad permiten ampliar el público objetivo al que podemos llegar porque nuestra aplicación puede estar en mayor número de dispositivos.

También, hemos aprendido que desde el punto de vista de **seguridad,** las pruebas que miden las posibles fugas de datos de nuestra aplicación son cada vez más importante, ya que, en la actualidad, los datos de los usuarios son un recurso muy requerido y tienen un alto valor para determinadas empresas para diferentes fines, por ejemplo, para campañas de publicidad dirigidas.

Resolución del caso práctico inicial

A la hora de buscar un posicionamiento web, las herramientas de búsqueda tienen en cuenta diferentes criterios para poder establecer el *ranking* de las páginas que quieren indexar y visualizar.

Obviamente, existen algunos criterios, como el pago, que nada tienen que ver con la calidad de la web.

Sin embargo, uno de los aspectos que también se consideran es la utilización de un código que permita mejorar la accesibilidad a diferentes colectivos. Llevando a cabo pruebas de accesibilidad web, podríamos tener en cuenta a un mayor público. De hecho, si cumplimos dichos criterios, podría mejorar directamente nuestro posible posicionamiento web.



Fig. 6. Tenemos que controlar las pruebas de accesibilidad.

/ 9. Bibliografía

Kaner, C.; Bach, J., y Pettichord, B. (2002). *Lessons learned in software testing: a context-driven approach.* New York: Wiley.
Beck, K. (2003). *Test-driven development: by example.* Boston: Addison-Wesley.
Beizer, B. (1995). *Black-box testing: techniques for functional testing of software and systems.* New York: Wiley.